

1. Práctica 1: *Bootloaders* y actualizaciones automáticas

1.1. Objetivos

- Entender el concepto de *bootloader* y programar uno.
- Aprender a comunicar con un módulo WiFi externo a la placa para la descarga de aplicaciones.
- Comprender los pasos a seguir para ceder el control de una aplicación a otra.
- Aprender cómo compilar las aplicaciones para que convivan con el *bootloader*.

1.2. Fundamentos teóricos

Hacer aplicaciones perfectas a la primera es algo muy complicado. Aún habiéndolas hecho, normalmente se requieren actualizaciones a lo largo del tiempo. Una vez hemos fabricado un dispositivo y está operativo, el enlace de depuración se suele perder, así que se requieren métodos más sofisticados para reprogramarlo. Uno de los más extendidos es el de programar un pequeño *bootloader*, o cargador, que se encarga además de actualizar el código principal si es necesario. En esta práctica programaremos un *bootloader* capaz de descargar aplicaciones via Wifi, grabarlas en la Flash, y por último cederles el control. Aprenderemos además cómo tenemos que compilar nuestras aplicaciones para que el *bootloader* las pueda cargar.

El *bootloader* es el primer programa que comienza a ejecutarse dentro del procesador. Su tarea, habitualmente, es realizar ciertas configuraciones iniciales, para posteriormente ceder el control a otro programa. Por ejemplo, en un ordenador “estándar”, el *bootloader* suele comprobar que los diferentes dispositivos están conectados correctamente y funcionando (tarjetas gráficas, memorias, pantalla, teclado, discos, etc), para posteriormente buscar distintos medios de arranque (USB, CD, Disco duro) y elegir automáticamente, o dar al usuario la opción, el modo de arranque del sistema.

Es desde este *bootloader* que uno puede configurar ciertos parámetros de muy bajo nivel del ordenador. Por ejemplo, encriptación de los discos (previa a la ejecución del sistema operativo), encendido automático a través de red, velocidad de los ventiladores...

Una vez se han realizado todas estas configuraciones y comprobaciones, el *bootloader* cede el control, generalmente, al sistema operativo (aunque en ocasiones hay varios niveles de *bootloading*). Éste ya gestionará el resto de tareas para su ejecución simultánea, gestionando los recursos del sistema.

En nuestro caso, no hay sistema operativo (aún), así que el *bootloader* simplemente cargará un programa sencillo. Para hacer algo más cercano a la realidad, haremos que nuestro *bootloader*, previo a la carga del programa, lo descargue automáticamente de un servidor si es que no estuviera ya en la memoria del procesador. Así imitaremos, en cierta forma, la capacidad de muchos dispositivos empotrados de actualizarse automáticamente, sin necesidad de ser configurados por los puertos de depuración.

Comprobará si la aplicación está ya instalada. De ser así, la lanzará directamente. Si no está, intentará descargarla de un servidor que configuraremos, por ejemplo, en nuestro teléfono. También daremos la opción de borrar la memoria donde alojaremos el programa, para poder

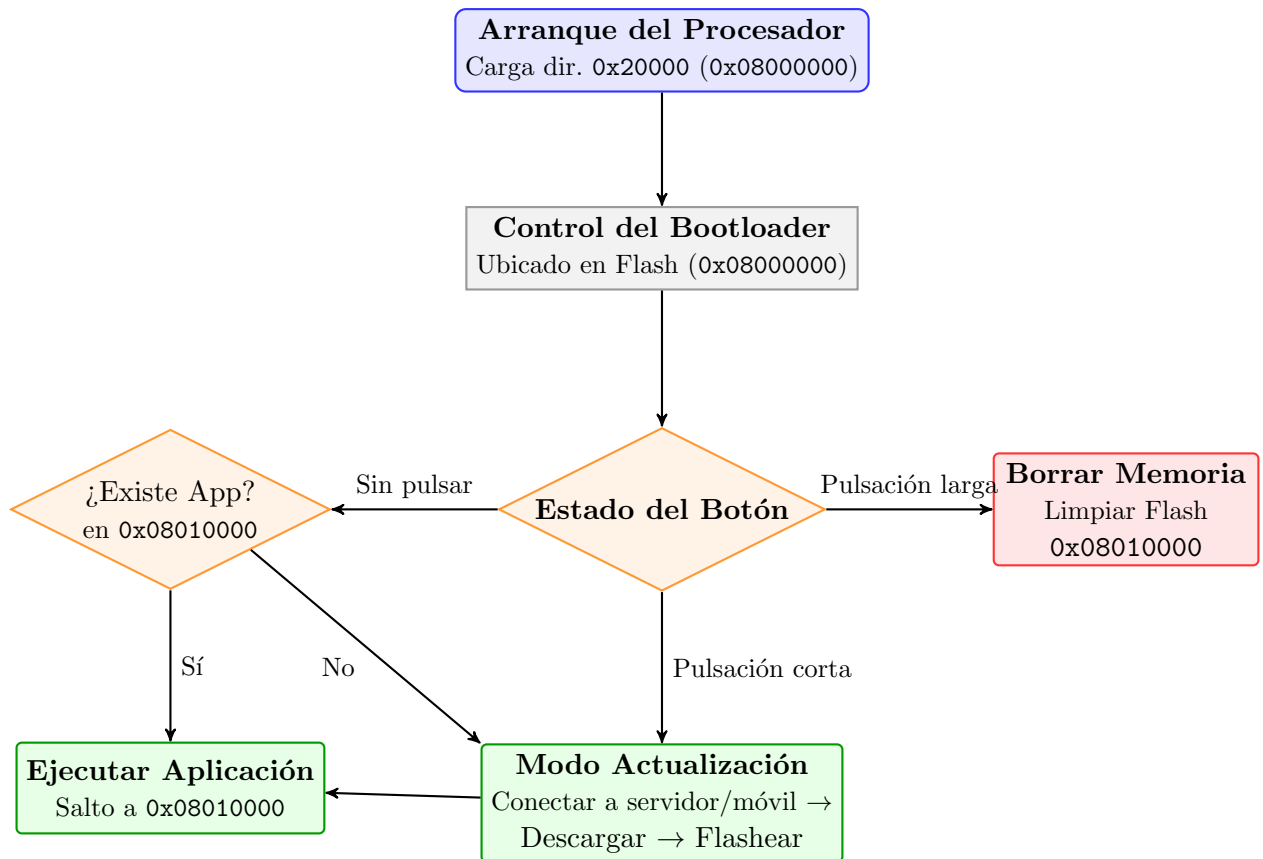


Figura 1: Secuencia de arranque de nuestra aplicación

probar el proceso varias veces sin necesidad de reprogramar la placa. El flujo de ejecución será el siguiente:

1. El procesador, desde su memoria de configuración, carga la dirección de arranque, en este caso la `0x20000` [rm0431 1.9](#). Nótese que esa dirección es un alias a la que nosotros utilizamos `0x08000000` [rm0431 3.3.1](#), ya que la **FLASH** se puede acceder desde dos rangos de direcciones diferentes.¹
2. El *bootloader* se encuentra allí, situado al principio de la flash `0x08000000` (memoria de arranque) y toma el control.
3. Si no hacemos nada, el *bootloader* consulta la dirección objetivo del programa (`0x08010000`) y verifica si existe un programa. En ese caso salta a ejecutarlo.
4. Si pulsamos un botón, el *bootloader* entrará en modo actualización, e intentará conectarse a internet para descargar, flashear, y por último ejecutar nuestra aplicación.
5. Si lo pulsamos aún más tiempo, borrará el contenido de la flash en la dirección `0x8010000`.

Cuando un *bootloader* cede el control, no puede hacerlo de cualquier manera. Debe asegurarse de que entrega el procesador a la aplicación invitada en un estado inicial aceptable. Por ejemplo,

¹Existe adicionalmente la posibilidad de programar dos direcciones diferentes de arranque, y controlar con un botón cuál de las dos se utiliza. Por defecto, la dirección de arranque será la de nuestro programa (*bootloader* en este caso), y en la otra, se encuentra otro *bootloader* diferente preprogramado por ST [rm0431 1.9](#). Para ver más detalles del *bootloader* de ST consultar [an2606](#)

deberá desactivar interrupciones, configurar correctamente la pila, e indicar la nueva dirección de inicio de programa al procesador. Un fallo en cualquiera de estos pasos podría suponer que el programa cargado no llegue a funcionar, generalmente porque empiece a ejecutarse en zonas de memoria restringidas.

Por otro lado, las aplicaciones también deben estar preparadas para su correcta ejecución. Normalmente, al preparar una aplicación “usual” en un procesador *bare-metal* como el nuestro, la compilamos con las opciones por defecto y esto es suficiente, ya que se ejecuta desde la dirección inicial de ejecución del procesador. Pero en este caso, en esa dirección tenemos el *bootloader*. Por tanto, debemos compilarla con la dirección de inicio objetivo en la cual la vaya a cargar el *bootloader*, asegurando así un correcto funcionamiento.

1.2.1. El módulo Wifi ESP-01S

Nuestro procesador no tiene manera de realizar comunicación inalámbrica, por tanto se utiliza un módulo externo ESP-01S. Este módulo permite, a través de una comunicación **UART** desde el procesador **STM32F723IE**, establecer una conexión WiFi y enviar o recibir datos a través de protocolos como HTTP y TCP, que serán vistos desde nuestra aplicación a través de la **UART**.

Nosotros lo veremos solo como usuarios y con un conjunto reducido de comandos. Puedes consultar la documentación completa en https://espressif-docs.readthedocs-hosted.com/projects/esp-at/en/release-v2.2.0.0_esp8266/AT_Command_Set/index.html.

1.3. Desarrollo de la práctica

1.3.1. Configuración previa al *bootloader*

Como hemos visto anteriormente, la idea es hacer un *bootloader* con diversas opciones (descargar app, ejecutar, borrar) que se seleccionarán de manera semi-automática en función del estado de la memoria, y el de un botón que podremos pulsar más o menos tiempo.

Para medir el tiempo, vamos a hacer uso de un periférico que hasta ahora no habíamos utilizado, el **SysTick** **pm0253 4.4**. Este periférico se utiliza para llevar una cuenta de los *ticks* que han pasado en el procesador. La cuenta de *tick* se incrementa cada cierto tiempo, dependiente de la velocidad del reloj (16MHz por defecto) y la configuración del periférico. Por ejemplo, podemos configurarlo para que cuente cada milisegundo. Así, podremos consultar en cualquier momento el valor de *tick* actual para saber cuánto tiempo ha pasado desde otro momento en el que realizáramos otra consulta. El código necesario se incluye a continuación:

```
1 #define SYSTICK_BASE_ADDR      (0xE000E010UL)
2
3 typedef struct {
4     volatile uint32_t CTRL;      // 0x00
5     volatile uint32_t LOAD;      // 0x04
6     volatile uint32_t VAL;       // 0x08
7     volatile uint32_t CALIB;     // 0x0C
8 } SysTick_TypeDef;
9
10 #define SysTick                ((SysTick_TypeDef *) SYSTICK_BASE_ADDR)
11
12 volatile uint32_t msTicks = 0;
13 void SysTick_Handler(void) {
```

```

14     msTicks++;
15 }
16
17 void SysTick_Init(void) {
18     // Default HSI clock is 16MHz on reset. 1ms tick = 16,000 cycles.
19     SysTick->LOAD = (16000U - 1U);
20     SysTick->VAL = 0U;
21     SysTick->CTRL = (1U << 2) | (1U << 1) | (1U << 0); // Processor Clock, Int
        Enable, Enable
22 }
23
24 void SysTick_Disable(void) {
25     SysTick->LOAD = 0U;
26     SysTick->VAL = 0U;
27     SysTick->CTRL = 0U;
28 }
29
30 uint32_t GetTick(void) {
31     return msTicks;
32 }
33
34 void Delay_ms(uint32_t ms) {
35     uint32_t start = GetTick();
36     while ((GetTick() - start) < ms);
37 }

```

Por otro lado, necesitamos configurar los periféricos a utilizar por nuestro bootloader. Estos son:

- El botón de usuario.
- La **UART** de depuración (ya configurada en prácticas anteriores)
- La **UART** de comunicación con el ESP-01S

Se proporcionan las siguientes funciones ya hechas para ello:

```

1 void Peripherals_Init(void) {
2     // Enable USARTs and their GPIOs
3     RCC_AHB1ENR->bits.GPIOCEN = 1;
4     RCC_AHB1ENR->bits.GPIODEN = 1;
5     RCC_APB1ENR->bits.USART5EN = 1; // comm with espressif
6     RCC_APB2ENR->bits.USART6EN = 1; // comm with computer (stlink)
7
8     /* --- USART6 Pin Configuration (ST-LINK VCP: PC6=TX, PC7=RX) --- */
9     GPIO_CONFIG(GPIOC, 6, GPIO_MODE_ALT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
10        GPIO_PUPD_PU, GPIO_AF8_USART6);
11     GPIO_CONFIG(GPIOC, 7, GPIO_MODE_ALT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
12        GPIO_PUPD_PU, GPIO_AF8_USART6);
13
14     /* --- UART5 Pin Configuration (ESP8266: PC12=TX, PD2=RX) --- */
15     GPIO_CONFIG(GPIOC, 12, GPIO_MODE_ALT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
16        GPIO_PUPD_PU, GPIO_AF8_USART5);
17     GPIO_CONFIG(GPIOD, 2, GPIO_MODE_ALT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
18        GPIO_PUPD_PU, GPIO_AF8_USART5);
19
20     /* --- Baud Rate Setup ---
21     * Default HSI Clock = 16 MHz. Target Baud Rate = 115200.
22     * USARTDIV = 16,000,000 / 115200 = 138.88 => BRR = 139 (0x8B)
23     */

```

```

20     UART_BRR_SET(USART6, 139);
21     UART_BRR_SET(USART5, 139);
22     // enable UARTS + TX + RX
23     UART_ENABLE_SET(USART6, 1);
24     UART_TX_MODE_SET(USART6, 1);
25     UART_RX_MODE_SET(USART6, 1);
26     UART_ENABLE_SET(USART5, 1);
27     UART_TX_MODE_SET(USART5, 1);
28     UART_RX_MODE_SET(USART5, 1);
29 }
30
31 void UserButton_Init(void) {
32     RCC_AHB1ENR->bits.GPIOAEN = 1;
33     GPIO_CONFIG(GPIOA, 0, GPIO_MODE_INPUT, GPIO_OTYPE_PP, GPIO_OSPEED_LOW,
34                 GPIO_PUPD_PD, 0);
35 }

```

1.3.2. Cesión del control a la aplicación

Para poder ceder el control a la aplicación, debemos hacer dos cosas. En primer lugar, ser capaces de comprobar que la aplicación existe y está descargada en la memoria. En segundo, cambiar el control del procesador a esa zona de memoria para su ejecución.

Comenzamos con la comprobación de existencia. Las aplicaciones, al compilarse, tienen una estructura muy concreta que se genera a partir del *Linker Script*. Este fichero (STM32F723IEKX_FLASH.ld), que viene creado ya de manera automática con nuestros proyectos (aunque podemos customizarlo), tiene esta pinta (se destacan aquí solo algunas partes):

```

1  // Punto de entrada del código al comenzar a ejecutarse
2  // Esta función llama luego a main
3  ENTRY(Reset_Handler)
4
5  // posición en memoria de la pila, justo al final de la memoria RAM
6  _estack = ORIGIN(RAM) + LENGTH(RAM);
7
8  // tamaños mínimos de pila y montón. Si no caben (por ser muy
9  // grande el programa), la compilación fallará
10 _Min_Heap_Size = 0x200;
11 _Min_Stack_Size = 0x400;
12
13 // direcciones, permisos y tamaño de las memorias
14 // importante que en la flash no se tienen permisos
15 // de escritura (por defecto)
16 MEMORY
17 {
18     RAM      (xrw)      : ORIGIN = 0x20000000,   LENGTH = 256K
19     FLASH    (rx)       : ORIGIN = 0x80000000,   LENGTH = 512K
20 }
21
22 // aquí comienzan a colocarse los ‘‘trozos’’ de programa
23 SECTIONS
24 {
25     // primero una tabla de vectores con diferentes direcciones
26     // para las distintas interrupciones del sistema
27     .isr_vector :
28     {
29         . = ALIGN(4);
30         KEEP(*(.isr_vector)) /* Startup code */
31         . = ALIGN(4);

```

```

32     } >FLASH
33
34     // luego se coloca el código
35     .text :
36     {
37         . = ALIGN(4);
38         *(.text)           /* .text sections (code) */
39
40         // continúa (ver archivo de referencia en cualquier proyecto)
41         // .....

```

Este linker script gobierna cómo el código se ordena y coloca en el binario final. Básicamente, va colocando todos los símbolos (por símbolo se entienden variables, funciones, etc) uno detrás de otro. Los símbolos se definen desde el código, al definir funciones o usar variables. Aunque nosotros no lo hacemos a mano, cada vez que definimos una función, se crean símbolos que comienzan por `.text`, o cada vez que definimos variables, símbolos que comienzan por `.data`. Eso es lo que coloca el linker posteriormente y de manera ordenada.

Ciertos símbolos especiales, como el código primero que se ejecuta, se definen desde el fichero `startup_stm32f723ieqx.s` (ya creado por defecto con cualquier proyecto). Aquí tenemos la función `Reset_Handler` que será la primera ejecutada al lanzar cualquier programa:

```

1  .section .text.Reset_Handler
2  .weak Reset_Handler
3  .type Reset_Handler, %function
4  Reset_Handler:
5  ldr    r0, =_estack
6  mov    sp, r0           /* set stack pointer */
7
8  // ... inicialización de diferentes secciones de memoria
9  // salto a la función principal.
10 bl main

```

Y cómo sabe el procesador que tiene que llegar aquí al iniciar? Miremos en otra zona del código de inicialización:

```

1  .section .isr_vector,"a",%progbits
2  .type g_pfnVectors, %object
3
4  g_pfnVectors:
5  .word _estack
6  .word Reset_Handler
7  // ... punteros a interrupciones

```

Es decir, si nos fijamos en el contenido `SECTIONS` de nuestro *linker script*, veremos que lo primero que coloca en el binario es `.isr_vector`. A su vez, esta sección tiene, en primer lugar, el valor inicial de la pila, y la dirección de comienzo `Reset_Handler`. El procesador ya es conocedor de esto [pm0253 2.1.3](#) y cargará ambos al resetearse.

Se ofrece a modo ilustrativo esta visualización del espacio de memoria de nuestra aplicación, construido desde el linker script y el código de inicialización:

Address (Flash)	Contents	Description
0x08000000	<code>.isr_vector</code>	Vector Table: Initial SP, Reset_Handler, ISR Handlers
0x08000000	<code>_estack</code> (0x20040000)	Initial Stack Pointer Value (Top of RAM)
0x08000004	<code>Reset_Handler</code>	Initial Execution Entry Address (Reset Vector)
0x08000008	<code>NMI_Handler</code>	Non-Maskable Interrupt Handler
0x0800000C	<code>HardFault_Handler</code>	Hardware Fault Exception Handler
0x08000010+	Core Exceptions	MemManage, BusFault, UsageFault, SVC, SysTick
0x08000040+	Peripheral IRQs	WWDG, PVD, RTC, Flash, RCC, EXTI, DMA, TIM, etc.
0x08000...	<code>.text</code>	Executable Program Code
...	<code>.rodata</code>	Constant Data, String Literals, Read-Only Variables
...	<code>.data</code>	Initialized global variables
...	...	Various code utilities
Address (RAM)	Contents	Description
0x20000000	<code>.data</code>	Initialized Global/Static Data (Copied from Flash)
...	<code>.bss</code>	Uninitialized Global Data (Zero-filled by startup)
...	Dynamic Heap	Minimum Reserved Size: 0x200 (512 Bytes) ↑ Grows Up
...	Unallocated	Available RAM for Dynamic Stack/Heap Overlap
0x2003FC00	Main Stack	Minimum Reserved Size: 0x400 (1024 Bytes) ↓ Grows Down
0x20040000	<code>_estack</code>	Highest Stack Address (End of RAM)

Observamos que, como habíamos dicho, el primer valor que se encuentra en nuestro programa, en `0x08000000` es la dirección de la pila, para a continuación tener en `0x08000004` la dirección inicial a la que saltar (es decir, un puntero). Así pues, si en una dirección de memoria encontramos esta estructura, o similar, podemos asumir que en ese lugar tenemos un programa guardado. Nuestra función de comprobación de existencia de programa puede ser, por ejemplo, la siguiente:

```

1 //Check for app presence at a given location
2 uint8_t IsApplicationValid(uint32_t address) {
3     uint32_t sp = *(volatile uint32_t *)address;
4
5     // Check if initial Stack Pointer points into valid SRAM range (0x200XXXXX)
6     if ((sp & 0x2FF00000) == 0x20000000) {
7         return 1;
8     }
9     return 0;
10 }

```

Ahora, una vez detectada una aplicación en una ubicación concreta de memoria, debemos conseguir saltar a ella. Lo más importante en este punto es saber que tenemos que dejar el procesador en un estado totalmente neutro, ya que de lo contrario, la nueva aplicación podría sufrir comportamientos inesperados. Imaginemos por ejemplo que no desactivamos las interrupciones o ciertos periféricos: el procesador seguiría ejecutando funcionalidad totalmente ajena a nuestra aplicación.

Por tanto, lo primero es desactivar cualquier cosa que hayamos utilizado. En nuestro caso, el *ticker* del sistema y los periféricos `GPIO` y `UART`.

```

1 void JumpToApplication(uint32_t address) {

```

```

2 // Disable any active interrupts (only SysTick in our case)
3 SysTick_Disable();
4
5 // Disable the peripherals we were using
6 RCC_AHB1ENR->bits.GPIOAEN = 1;
7 RCC_AHB1ENR->bits.GPIOCEN = 1;
8 RCC_AHB1ENR->bits.GPIODEN = 1;
9 RCC_APB1ENR->bits.USART5EN = 1; // comm with espressif
10 RCC_APB2ENR->bits.USART6EN = 1; // comm with computer (stlink)

```

Por otro lado, tenemos que reubicar también el puntero de la pila para que se ubique al principio del nuevo rango requerido. De lo contrario, estaríamos trabajando con la pila en direcciones de memoria desconocidas, lo cual puede traer fallos de acceso a memoria o, aún peor, sobrescribir nuestro nuevo código si hubiera solape. El valor inicial del **SP** se extrae de la dirección inicial del programa:

```

1 // Set Main Stack Pointer (MSP)
2 __asm volatile ("msr msp, %0" : : "r" (*(volatile uint32_t *)address) : );

```

Ahora ya está todo listo para saltar a la función inicial (que recordemos es el **Reset_Handler** del archivo de inicialización **startup_stm32f723ieqx.s**). Como su dirección inicial se encuentra en la segunda palabra del binario (**address + 4**), la extraemos de ahí, y la convertimos a un tipo de función **void** antes de llamarla.

```

1 // Branch to reset handler of target firmware
2 typedef void (*pFunction)(void);
3 uint32_t JumpAddress = *(volatile uint32_t *)(address + 4);
4 pFunction JumpToApp = (pFunction)JumpAddress;
5 JumpToApp();

```

A partir de aquí, habremos perdido completamente el control del programa, y toda la ejecución queda a merced de la aplicación objetivo.

1.3.3. La memoria Flash

Nuestro código se almacenará en la memoria Flash del procesador. Esta memoria tiene la capacidad, al contrario de la RAM, de retener el contenido incluso tras apagarse y encenderse el procesador. Sin embargo, su uso por el programador está restringido (básicamente para evitar accidentalmente sobrescribir el código) así que tenemos que permitir los accesos en ella si queremos hacerlo. Echa un vistazo a la documentación de la **FLASH** en **rm0431 3.7.2** y completa el código de desbloqueo de la flash:

```

1 /* Flash Keys */
2 #define FLASH_KEY1          0x45670123U
3 #define FLASH_KEY2          0xCDEF89ABU
4 void Flash_Unlock(void) {
5     // ... completar si la flash está bloqueada (CR)
6     // ... desbloquear con la secuencia de claves en KEYR
7 }

```

Observa también la función **ESP01S_DownloadAndFlash** y verás que el flujo de programación es, a grandes rasgos, el siguiente:

```

1 Flash_Unlock();
2 Flash_ClearErrors();

```

```

3  for (bytes to write)
4      Flash_WriteByteRaw(current_flash_addr++, byte);
5  Flash_Lock();

```

Es decir, nos aseguramos de que la protección de la flash solo se quita en el fragmento de código que flashea el binario descargado, y en ningún otro sitio, para evitar errores.

1.3.4. Programación del *bootloader*

Ahora, con el archivo descargado y las funciones auxiliares de salto a programa, debemos programar el *bootloader*. Básicamente vamos a crear un código `main.c` que ceda el control a una región de memoria específica (en concreto, la `0x08010000` como vimos anteriormente). Completa el siguiente código para ello, añadiendo además los mensajes de depuración (Ver práctica 3) que consideres oportunos. Utiliza todas las funciones programadas anteriormente.

```

1  int main(void) {
2      // ... completar
3      // ... inicializar systick, periféricos y botón de usuario
4
5      uint8_t app_valid = IsApplicationValid(ADDR_FLASH_SECTOR_4);
6      uint8_t button_pressed = GPIO_PIN_READ(GPIOA, 0);
7      // Boot decision
8      // ... completar
9      // ... si el botón no está pulsado, comprobar si la app es válida y saltar a
10     ella
11     // ... si el botón está pulsado, comprobar de nuevo tras 2 segundos
12     // ... si sigue pulsado, en ese caso borrar la flash
13     // ... en cualquier otro caso, continuar con la descarga de la app
14
15     // Initialize ESP8266 device
16     if (!ESP01S_Init()) {
17         while (1);
18     }
19
20     // Connect Wi-Fi
21     if (!ESP01S_ConnectWiFi()) {
22         while (1);
23     }
24
25     // We must erase before flashing new stuff, since bits only
26     // change from 1 to 0 and not vice versa when flashing.
27     // This ensures initial state is 0xffffffff
28     Flash_EraseSector(FLASH_TARGET_SECTOR);
29
30     // Download over TCP and Flash directly
31     if (!ESP01S_DownloadAndFlash()) {
32         while (1);
33     }
34
35     JumpToApplication(ADDR_FLASH_SECTOR_4);
36 }

```

1.3.5. Compilación de la aplicación

Con esta práctica se proporciona un fichero binario compilado de ejemplo. No obstante, si queremos compilar el nuestro propio (con otro programa) tenemos que seguir unos pasos para asegurar que funcione.

Lo primero es ceder el control completamente de forma correcta. Hasta ahora, lo “único” que hemos hecho ha sido saltar a la dirección de la función inicial **Reset_Handler** de la aplicación objetivo. Pero, y todas las interrupciones? Si ahora activásemos una interrupción, saltaría a la tabla de interrupciones de la aplicación original, fallando por completo el código. Como vimos anteriormente, la aplicación tiene al comienzo una tabla de vectores que indican las direcciones de memoria de las diferentes interrupciones y código de arranque. El Bloque de Control del Sistema **SCB** (**pm0253 4.3**) guarda en su registro **VTOR** la dirección inicial de dicha tabla (es decir, la dirección inicial de nuestro código). Así pues, debemos cambiarla para reubicar todas las interrupciones de una sola tacada. Esto lo podemos hacer redefiniendo la función **System_Init** (definida como **.weak** precisamente para que nosotros la redefinamos y le demos funcionalidad si es necesario).

```
1 void SystemInit(void) {  
2     SCB->VTOR = 0x08010000;  
3 }
```

Por otro lado, tendremos que indicar en el linker script **STM32F723IEKX_FLASH.ld** el cambio de dirección inicial:

```
1 FLASH      (rx)      : ORIGIN = 0x8010000,   LENGTH = 256K
```

Finalmente, activamos la generación del archivo binario desde **C/C++ Build >Settings >MCU/MPU Post build outputs >Convert to binary file (-O binary)**.

Al compilar, se nos generará ahora un archivo **.bin** que es el que habrá que flashear.

1.3.6. Configuración del servidor de descarga

Llegados a este punto, deberíamos tener un bootloader funcional que... no puede hacer nada pues no tiene un servidor al que conectarse. Necesitamos hacer dos cosas:

1. Asegurarnos de que el módulo ESP-01S está conectado a nuestra placa.
2. Configurar un servidor para que se descargue el archivo binario.

Lo primero es relativamente sencillo, simplemente hay que colocar el módulo en el conector CN14 que se encuentra en la placa (conector con 8 pines huecos), en la dirección que indica la marca blanca inferior.

Respecto a lo segundo, se puede hacer de muchas maneras. La más cómoda es hacerlo desde nuestro propio teléfono, habilitando un punto inalámbrico que actúe como entrada a la red para nuestro programa. Asegúrate de configurar correctamente la conexión creando el archivo **ESP01S_config.h**

```

1 #define WIFI_SSID      "<nombre de tu red wifi>"
2 #define WIFI_PASSWORD "<password de tu red wifi>"
3 #define SERVER_IP      "<ip de tu servidor>"
4 #define SERVER_PORT    <puerto de tu servidor>
5 #define FILE_PATH      "<nombre del fichero>"

```

Lo siguiente es abrir un servidor `http` en nuestro teléfono que sirva el binario `hello.bin` o cualquier otro que hayamos creado a mano.

- En Android, podemos hacerlo descargando la aplicación SHTTPS <https://play.google.com/store/apps/details?id=com.phlox.simpleserver&hl=es> y configurándola como vemos en la Figura 2. Únicamente tendremos que iniciar el servidor, poner la `Root Folder` apuntando a donde tengamos guardado el archivo binario de la aplicación, y seleccionar la interfaz de nuestra red inalámbrica (normalmente llamada `ap_XXX`) para que nos de la IP que tendremos que poner en la configuración anterior.
- En IOS, hay aplicaciones similares como <https://apps.apple.com/es/app/simple-server-http-server/id6443893597> o <https://apps.apple.com/es/app/share-server/id6754194116>.

Y listo! Ahora nuestra aplicación debería ser capaz de conectarse al punto WiFi de nuestro teléfono, descargar el binario, flashearlos, y lanzarlo a ejecución! Asegúrate de poner mensajes de depuración si tienes problemas, para ver dónde pueda haber dificultades.

1.4. Para hacer más

En ocasiones, en lugar de tener una aplicación *bootloader* que se encarga de descargar el binario actualizado, se fusionan ambas funcionalidades en una sola (*bootloader* + app). Así, el propio *bootloader* también es actualizado. Para ello, se reservan dos regiones de flash diferentes, *A* y *B*. La aplicación *A*, cuando detecte una actualización (a través de WiFi por ejemplo), la escribirá en *B*, y configurará el procesador para arrancar desde ahí la próxima vez. Y viceversa, si la *B* es la que está en juego, actualizará *A*. Así, incluso si durante el proceso de actualización fallara algo, siempre habrá una de las dos aplicaciones funcional. Investiga cómo se haría este proceso en nuestro procesador **STM32F723IE**.

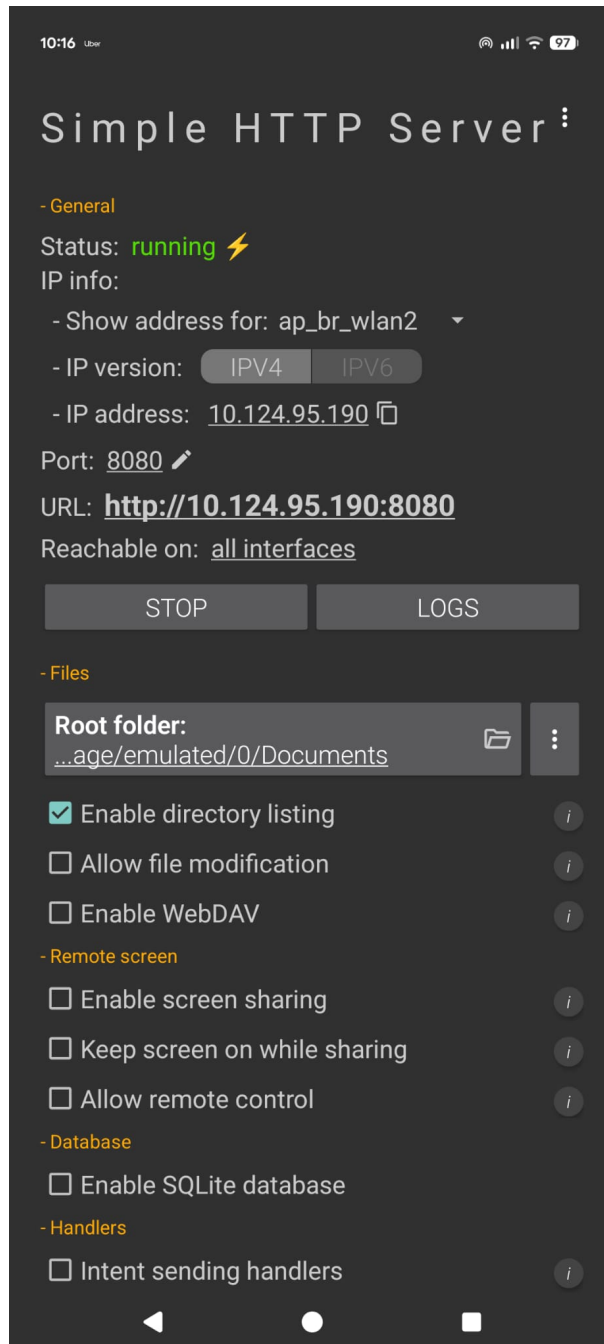


Figura 2: Configuración SHTTPS Android